

PostgreSQL Performance Playbook (with Examples)

An operator-focused guide for diagnosing, stabilizing, and optimizing PostgreSQL in production—with workload-specific strategies and cloud provider guidance.

1. Tailoring Optimization: Read-Heavy vs Write-Heavy Workloads

Read-Heavy Systems (analytics, dashboards, APIs)

- Aggressive indexing on WHERE / JOIN / ORDER BY columns
- Higher `shared_buffers` and `effective_cache_size`
- Favor covering indexes to reduce heap access
- Use replicas for read scaling and offloading
- Cache at application or query layer where possible

Write-Heavy Systems (event ingestion, queues, logs)

- Minimize indexes—each write updates every index
- Batch inserts and updates
- Tune autovacuum aggressively
- Short transactions only
- Consider UNLOGGED tables for ephemeral data

2. Top 10 PostgreSQL Performance Mistakes Seen in Production

- Over-indexing tables without measuring read vs write cost
- Ignoring autovacuum until bloat causes outages
- Running EXPLAIN instead of EXPLAIN ANALYZE during diagnosis
- Using OFFSET pagination on large datasets
- Storing frequently queried fields inside JSONB without indexes
- Letting long-running transactions block vacuum
- Assuming sequential scans are always bad
- Using default memory settings at scale
- Failing to monitor pg_stat_statements
- Optimizing rare queries instead of the hot path

3. PostgreSQL Incident Response & Performance Playbook

- Step 1: Identify impact—latency, errors, throughput
- Step 2: Capture top queries via `pg_stat_statements`
- Step 3: Run `EXPLAIN ANALYZE` on offending queries
- Step 4: Check locks, waits, and long transactions
- Step 5: Validate disk, CPU, and memory saturation
- Step 6: Apply lowest-risk mitigation first (index, `LIMIT`, kill query)
- Step 7: Measure improvement and document findings

4. Cloud-Specific PostgreSQL Considerations

AWS RDS / Aurora PostgreSQL

- Use Performance Insights and Enhanced Monitoring
- Be aware of I/O credits and burst balance
- Aurora favors read scaling; writes still bottleneck at writer node
- Tune parameter groups—defaults are conservative

Google Cloud SQL for PostgreSQL

- Watch CPU-ready and storage latency metrics
- Vertical scaling often outperforms premature sharding
- Plan maintenance windows—restarts are not optional

Azure Database for PostgreSQL

- Retire Single Server—use Flexible Server
- Understand IOPS limits tied to disk size
- Monitor connection limits aggressively